深度学习与文本分析

2026年3月

关于我

教师：郑杰鑫

- 管理科学与信息系统系助理教授
- 光华管理学院2号楼454室
- jiexinzheng@gsm.pku.edu.cn

研究领域

- 金融科技
  - 处理、分析大规模金融数据
- 智能体系统、AI4S
  - 智能体设计；基于智能体的知识发现
- 人与AI的交互
  - 如何部署AI系统以最大化其收益（e.g., 提高生产效率）及最小化其风险（e.g., 算法操纵）

课程定位和目标

- 使非计算机专业的同学能够快速理解和应用深度学习技术在自然语言处理领域的方法和工具。
- 培养学生对于文本数据的处理和分析能力，以及使用PyTorch深度学习框架进行自然语言处理任务的能力。
- 提高学生对大语言模型的理解和应用能力。
- 基于大语言模型搭建具有实际价值的系统

课程定位和目标

- 学生背景要求
  - 拥有基础的Python编程能力
  - 入门级别的数学知识
    - 基础线性代数
    - **矩阵的加减乘除、变化**
  - 机器学习的基础知识
    - 理解什么是训练集、测试集；分类任务、回归任务……
    - **反向传播算法**
  - 如果缺乏知识基础……
    - 也没关系。非常简单，可以快速自学。
  - 这是一门强调应用的课程，可以是你第一门人工智能相关的课程。

课程定位和目标

- 与其他机器学习相关课程的关系
  - 关注自然语言处理
  - 代码开发基于Pytorch

- 你会/不会学到什么……
  - 会：
    - 目前常用的自然语言处理相关的深度学习模型的基本概念和应用；基于特定的数据集、特定的任务需求对模型进行调整；
  - 不会：
    - 模型底层的数学、实现方法；最新最前沿的模型（近一两年）

课程安排

时间	课程内容	作业安排
第1周	课程内容简介；模型构建的注意事项；HuggingFace简介	作业1发布
第2周	文本数据的获取、清理和储存；自然语言处理的常见任务、应用场景	
第3周	从神经网络到循环神经网络（经典的RNN, LSTM结构）	作业2发布
第4周	大语言模型的基本概念和发展历程；Attention机制和Transformer结构	
第5周	Attention机制和Transformer结构	
第6周	预训练（pre-train）和微调（fine-tune）	作业3发布
第7周	通过RLHF（reinforcement learning from human feedback）训练聊天机器人	
第8周	提示工程（prompt engineering）；调用大语言模型的api服务完成常见的文本分析任务	作业4发布
第9周	LoRA; 微调与蒸馏大语言模型	
第10周	基于大语言模型构建的智能体系统	作业5发布
第11周	Agentic AI原理及其常见应用	
第12周	基于大语言模型的应用开发（RAG, MCP, skills等）	作业6发布
第13周	设计科学：为创意任务设计智能体系统；基于大语言模型的知识发现	
第14周	期末论文汇报	
第15周	期末论文汇报	

课程安排

助教

- 宋香凝　2401111086@stu.pku.edu.cn
- 布桐　2401111062@gsm.pku.edu.cn

- 负责签到、答疑、批改作业、成绩录入

群聊: 深度学习与文本分析课程群
2026

该二维码7天内(3月8日前)有效，重新进入将更新

课程大纲

- 考核：

课堂参与	15%
平时作业*6	60%
前沿专题研讨	25%

允许使用AI工具完成作业。鼓励人机协作！会提供GPU算力。

不签到；每节课随机点3~5个名。

每个小组人数上限四人

课程安排

作业内容：

- 作业1：注册deepseek API; 使用Coding Agent和deepseek API完成基础的文本分析任务
- 作业2：文本数据清理；构建基础的用于文本分类的神经网络
- 作业3：微调预训练好的BERT模型，完成文本分类任务
- 作业4：提示词工程
- 作业5：蒸馏并训练一个特定领域的大语言模型
- 作业6：搭建多智能体系统

课程大纲

- **考核：**
  - **作业**
  - 体验自己训练AI模型、打造AI应用。锻炼学生的工程实现能力。
  - 过程中可能会有一些负面体验：
    - 配置环境、训练模型很花时间
    - 训练结果难以控制
    - 莫名其妙的bug……
    - ……
    - 这些负面体验像是副作用很小的疫苗，它将帮助你在未来面对更大的挑战。

课程大纲

- **考核：**
  - **第 14、15 周 前沿专题研讨 (小组分享)**
  - 任务介绍：在本次前沿专题研讨活动中，各小组将从 Neurips、ACL、NAACL、HCI 等前沿学术会议中挑选一篇自己感兴趣的文本处理或大语言模型相关文章进行深入研究，并在课堂上进行分享。各小组需要对所选文章进行全面、深入的理解，包括但不限于文章的研究背景、科研动机、实现方案等关键部分。每个小组的分享时间限定为 20 分钟，之后将留出 5~10 分钟的时间供教师和其他同学提问和交流。

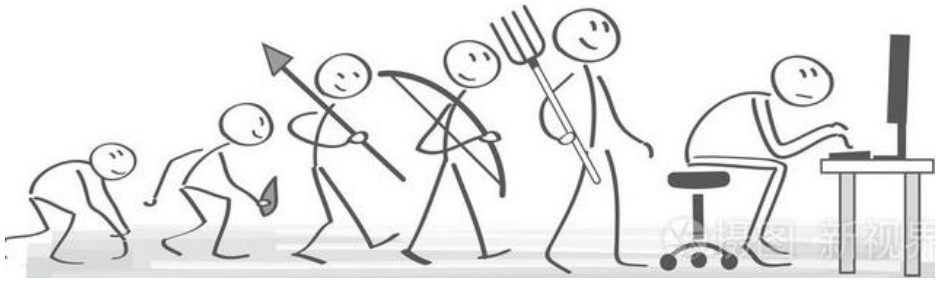
深度学习与自然语言处理

- 什么是自然语言处理？
  - 自然语言~人类语言
    - 区别于人工语言（比如程序语言）
  - 自然语言处理包括语音识别、自然语言理解、自然语言生成、人机交互以及所涉及的中间阶段
    - 是人工智能和计算机科学的子学科。
    - 本门课程主要关注基于文本数据的自然语言处理

  - 基础技术：词法分析、句法分析、实体识别、语义分析、语言模型
  - 应用：文本信息提取、舆情分析、智能客服、搜索引擎

深度学习与自然语言处理

- 经管学生也能玩转深度学习
  - 时代变了！



强大的工具、社区使得深度学习模型的使用、开发难度急速下降！！

CONDA® python PyTorch Hugging Face Ollama docker

深度学习与自然语言处理

- 经管学生也能玩转深度学习
  - 高度封装的代码

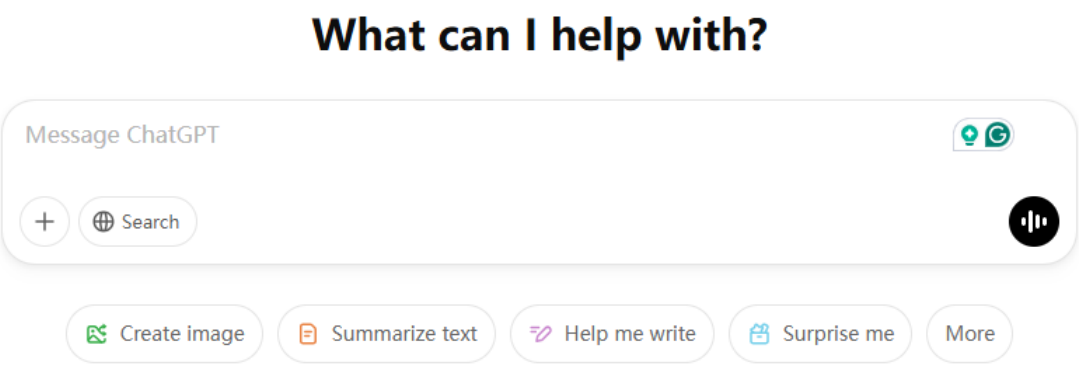
```python
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import pipeline

finbert = BertForSequenceClassification.from_pretrained('yiyanghkust/finbert-tone',num_labels=3)
tokenizer = BertTokenizer.from_pretrained('yiyanghkust/finbert-tone')

nlp = pipeline("sentiment-analysis", model=finbert, tokenizer=tokenizer)

# 需要预测情感的句子
sentences = ["there is a shortage of capital, and we need extra financing",
             "growth is strong and we have plenty of liquidity",
             "there are doubts about our finances",
             "profits are flat"]
results = nlp(sentences)
#LABEL_0: neutral; LABEL_1: positive; LABEL_2: negative
for d in results:
    print(d)
```

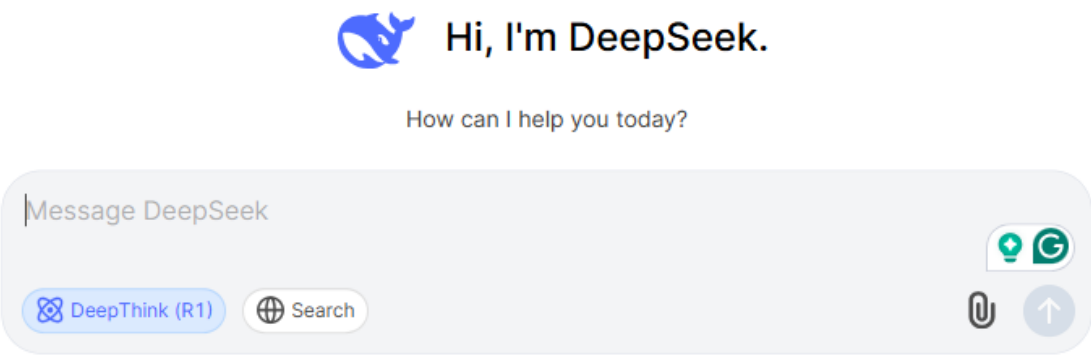
```
{'label': 'Negative', 'score': 0.9966173768043518}
{'label': 'Positive', 'score': 1.0}
{'label': 'Negative', 'score': 0.9999710321426392}
{'label': 'Neutral', 'score': 0.9889442920684814}
```

使用FinBERT模型分析金融文本的情感。

深度学习与自然语言处理

- 经管学生也能玩转深度学习
  - 与大语言模型交流





深度学习与自然语言处理

- 经管学生也能玩转深度学习
  - 使用大语言模型的API服务

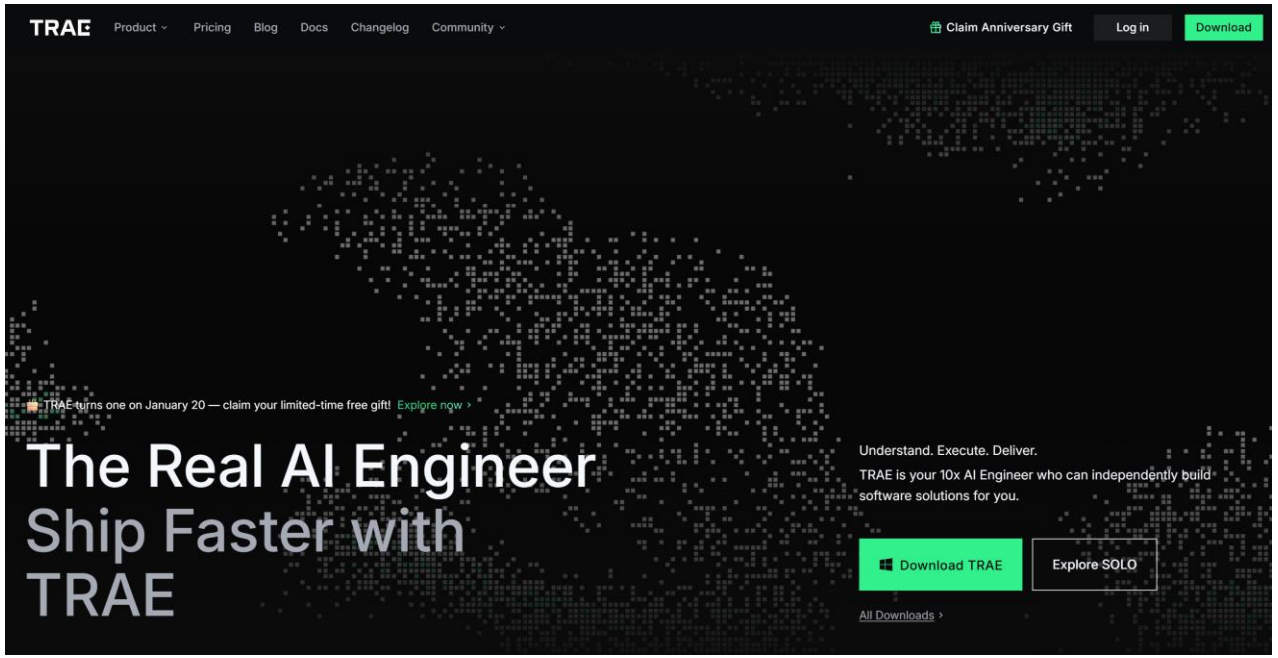
```python
response_result = client.chat.completions.create(model="gpt-4o-mini",
                                                 messages=[{"role": "user", "content": 'who are you?'}])
response_content = response_result.choices[0].message.content
print(response_content)
```

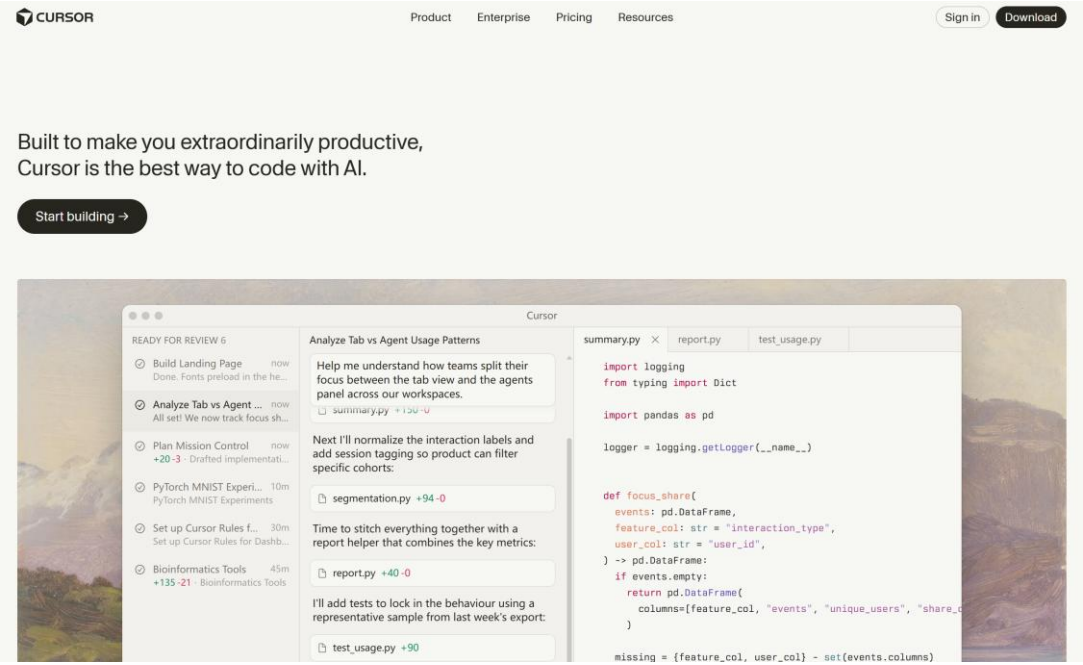
```
I am an AI language model created by OpenAI, designed to assist with a wide range of questions and tasks. I can provide information, answer queries, help with writing, and engage in general conversation. How can I assist you today?
```

通过API调用大语言模型

深度学习与自然语言处理

Coding agent的出现和发展，极大降低了开发的难度。

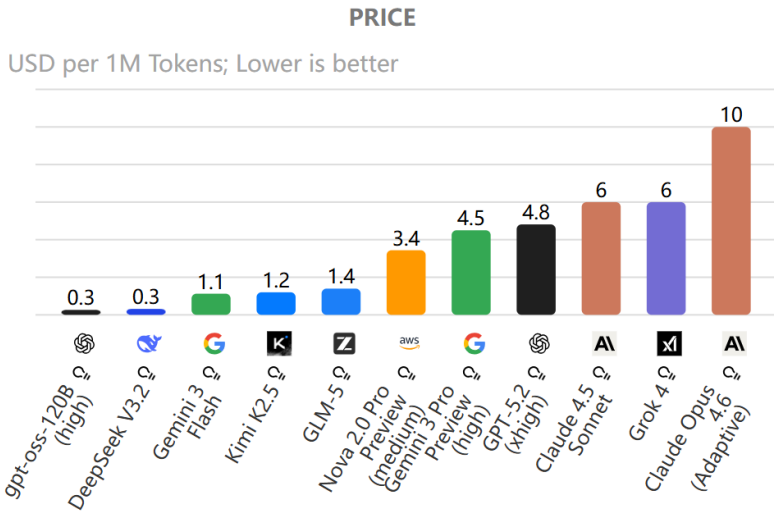




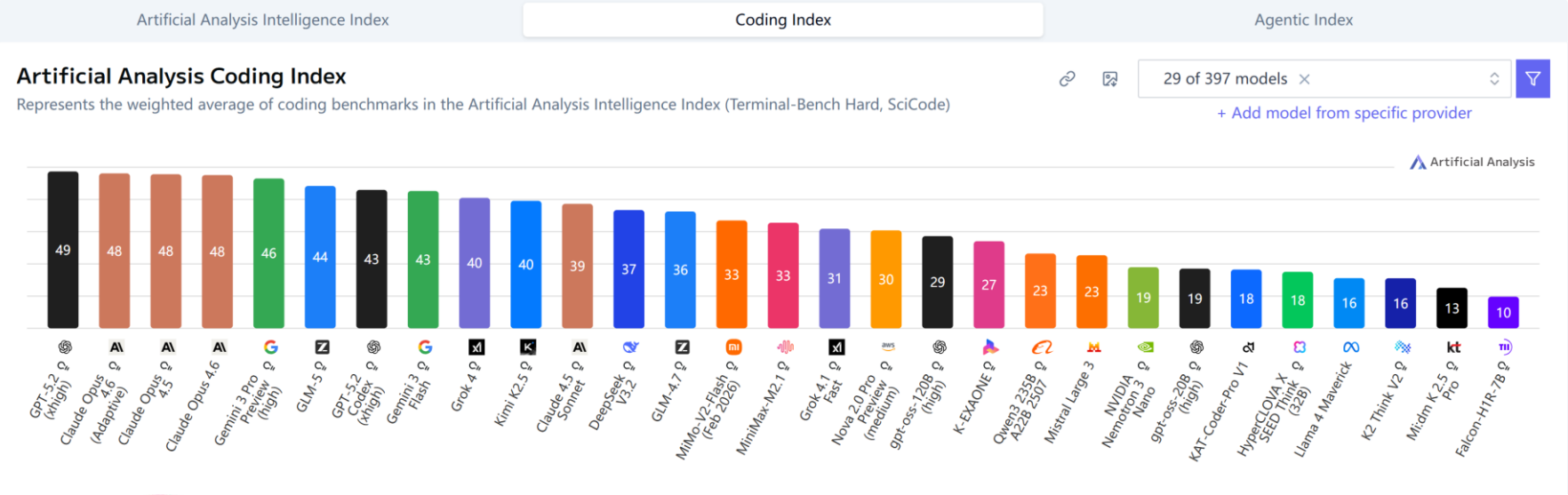
务必熟练使用**coding agent!**

深度学习与自然语言处理

国产大语言模型迅速发展



在权威榜单**Artificial Analysis**中，**GLM-5**位居全球第四、开源第一



我的深度学习经历

- 自学两个月，参加竞赛

比赛日程

11.06-11.22	**报名 参赛选手通过「JDD—2017京东金融全球数据探索者大赛」官网进行报名，报名成功后即可进入选拔赛。** - 11.22 16:00 截止报名
11.09-12.12	**选拔赛 根据赛题通过机器评分选出20名参赛选手进入总决赛** - 11.09 00:00 开放数据、结果提交和A榜评测 - 12.08 24:00 关闭选拔赛A榜 (A榜一天可以提交1次，以提交的最优结果为准) - 12.09 00:00 开放B榜提交，并进行小号作弊和算法有效性排查(B榜一天可以提交2次，以提交的最优结果为准) - 12.11 24:00 关闭选拔赛B榜 - 12.12 18:00 根据B榜成绩公布入围线下决赛的20名参赛选手名单，前20名的分布为（若某道赛题报名队伍较少时，经组委会讨论及技术评判后，报名队伍可不按照以下公式计算，直接入选）： 赛题1入选人数：赛题1报名队伍数/算法组总报名队伍数*20 赛题2入选人数：赛题2报名队伍数/算法组总报名队伍数*20 赛题3入选人数：赛题3报名队伍数/算法组总报名队伍数*20 赛题4入选人数：赛题4报名队伍数/算法组总报名队伍数*20
12.15-12.17	**总决赛 20名参赛选手线下封闭48小时线下总决赛** - 12.15 16:00-18:00 京东总部大楼集合，明确赛前相关规则，赛前辅导。 - 12.15 20:00 总决赛开始，京东金融将开放更多竞赛数据，并安排京东金融数据导师参与辅导。 决赛入围选手升级线下赛用代码，对选拔赛开发的数据结果进行内测及调整修正。提交内容包括数据结果和代码，每位参赛选手有三次提交机会，最终结果以最优提交的为准。 选手需自带电脑。 - 12.17 10:00 结果提交截止 - 12.17 10:00-12:00 选手阐述环节 - 12.17 15:00-17:00 观摩商业组路演决赛 - 12.17 17:00-17:30 颁奖仪式

猪脸识别

上传文件

算法组 · 比赛进行中 · 当前参赛人数 1386 人 · 比赛截止日期 2017-12-17

赛事介绍 赛题与数据 常见问题 **排行榜** 提交结果

选拔赛B榜	排名	用户昵称	得分	提交时间
选拔赛B榜	我的成绩 15	jd_7c91df0a98c02	0.290085	2017-12-10 12:11:22

赛事总奖金200+万元

我的深度学习经历

- 第二次比赛经历

FashionAI全球挑战赛—服饰属性标签识别

参加时间：2018-03-05

参加形式：中科9¾站台清理工

第1赛季排名：19/2950 查看证书

第2赛季排名：3/2950 查看证书

举办方：

Alibaba Group 阿里巴巴集团 | The Hong Kong Polytechnic University 香港理工大学 | The Textile Institute

已结束

我的深度学习经历

- 第三次比赛经历

爱奇艺多模态视频人物识别挑战赛

本比赛基于海量的爱奇艺综艺和影视剧视频片段数据集，进行多模态人物识别任务的挑战研究

举办方:　　报名截止日期：2018-09-17

总奖金 ¥350,000

397 支队伍已参加

竞赛介绍　竞赛数据　讨论组　常见问题　排行榜　我的提交　已参赛

名次	队伍名称	最优分数	所在组织	提交时间
1	____WitcheR	0.8865	InfiniVision	2018-10-13
2	偏偏我是猫	0.8766	百度	2018-10-15
3	周同俱乐部	0.8519	周同科技_北工大	2018-10-15
4	炸天	0.8505	香港科技大学	2018-10-15
5	emmm...	0.8252	南京大学	2018-10-12
6	喜欢爬山的北野琇莹	0.8091	VPP	2018-10-15
7	188****4098_m1097	0.8087	重庆中科绿色研究院智能安全中心	2018-10-15
8	dianxin55	0.8076	极涟科技	2018-10-14
9	高贵的宇文玄雅	0.8075	北京邮电大学	2018-10-14
10	陈少萌萌哒	0.8027	平安科技	2018-10-15

我的深度学习经历

- 学习体会
  - 并不难
  - 需要大量的练习。持之以恒
  - 年轻就是资本！

- 社区、技术的发展降低了学习的难度
  - 专注于理解。
    - Coding agent实现了极大的提效。人机协作中，我们需要严格把关。
  - 代码实现 VS. 寻找、评估、优化解决方案。

我的深度学习经历

- 从纯算法、模型的研发转向AI在商业领域的应用

Measuring Information Processing Cost with Language Predictability: A Deep Learning Approach

JOURNAL OF MANAGEMENT INFORMATION SYSTEMS
2024, VOL. 41, NO. 1, 178–205
https://doi.org/10.1080/07421222.2023.2301176

Routledge
Taylor & Francis Group

Check for updates

The Effects of Sentiment Evolution in Financial Texts: A Word Embedding Approach

Jiexin Zheng[a], Ka Chung Ng[b], Rong Zheng[a], and Kar Yan Tam[a]

[a]Department of Information Systems, Business Statistics and Operations Management, School of Business and Management, Hong Kong University of Science and Technology, Clear Water Bay, Kowloon, Hong Kong; [b]Department of Management and Marketing, Faculty of Business, Hong Kong Polytechnic University, Hung Hom, Kowloon, Hong Kong

ABSTRACT
We examine the evolutionary effects of sentiment words in financial text and their implications for various business outcomes. We propose an algorithm called **Word List Vector** for **S**entiment (WOLVES) that leverages both a human-defined sentiment word list and the word embedding approach to quantify text sentiment over time. We then apply WOLVES to investigate the evolutionary effects of the most popular financial word list, Loughran and McDonald (LM) dictionary, in annual reports, conference calls, and financial news. We find that LM negative words become less negative over time in annual reports compared to conference calls and financial news, while LM positive words remain qualitatively unchanged. This finding reconciles with existing evidence that negative words are more subject to managers' strategic communication. We also provide practical implications of WOLVES by correlating the sentiment evolution of LM negative words in annual reports with market reaction, earnings performance, and accounting fraud.

KEYWORDS
financial texts; market sentiment; word embedding; sentiment evolution; textual analysis; financial communication; word corpora; financial word lists

Abstract

propose a novel text information processing cost measure and use it to investigate the
nation content of corporate disclosures. Grounded on the language predictability theory
c and cognitive science research, this new measure calcu
choices from the regular language patterns learned
ssing difficulty of human. We implement this measure wit
ge text dataset of firms' 10-K annual reports. We find th
ss predictable in their MD&As. Besides, lower languag
reports is associated with lower future earnings perfor
gher volatility in post-filing returns, and lower accuracy ar
. Overall, our results suggest that language predictabi
market participants and thus an effective measure of info

Management and Organization Review (2024), **20**, 161–203
doi:10.1017/mor.2024.6

CAMBRIDGE UNIVERSITY PRESS

ARTICLE

Firms' Rhetorical Nationalism: Theory, Measurement, and Evidence from a Computational Analysis of Chinese Public Firms

Lori Qingyuan Yue[1], Jiexin Zheng[2], and Kaixian Mao[3]

[1]Columbia Business School, Columbia University, New York, NY, USA, [2]Department of Information Systems, Business Statistics, and Operations Management The Hong Kong University of Science and Technology, Hong Kong SAR, China, and [3]School of Labor and Human Resources, Renmin University of China, Beijing, China
Corresponding author: Kaixian Mao (kxm@ruc.edu.cn)

(Received 3 July 2022; accepted 3 December 2023)

Abstract
In this article, we develop a computational measure of firm-level rhetorical nationalism. We first review the literature and develop a four-dimensional theoretical framework of nationalism relevant to firms: national pride, anti-foreign, dominant agenda (national revival), and corporate role. We then use machine-learning-based text analysis of over 41,000 annual reports of Chinese public firms from 2000 to 2020 and identify a dictionary of words for each dimension. Using a weighted ratio of nationalism-related words, we describe the overall picture of Chinese public firms' rhetorical nationalism and provide the first empirical evidence regarding rising rhetorical nationalism among Chinese firms. Firms' demonstration of rhetorical nationalism is related to both strategic and socialization factors; firms that are state-owned enterprises, older, larger, more profitable, consumer-facing, with more individual investors, and lower sales from overseas demonstrate a higher level of nationalism. Firms that demonstrate more rhetorical nationalism also have a better future financial return. Our study provides a theoretical framework for the organizational study of nationalism and a new measure for firms' rhetorical nationalism, and demonstrates that rising rhetorical nationalism among Chinese firms is more strongly driven by firms' motivations to appeal to domestic investors and consumers than to obtain government subsidies. Our dataset is publicly available at: https://sites.google.com/view/firms-rhetorical-nationalism

我的深度学习经历

- 近期研究重点（关注大语言模型在商业领域的应用）
  - 金融欺诈检测
  - 多智能体系统设计
  - AI for Science

如果你对深度学习+金融市场感兴趣……

- 项目机会：
  - 基于深度学习的端到端的高频交易模型
- 提供资源：
  - 数据支持
  - 硬件支持
  - 交易系统支持，专注于模型研发
  - 考核通过后有补贴
- 预期：
  - 较强的数理能力；自我驱动力
- 目标：小而精的团队；

本节课内容

- 深度学习框架及PyTorch入门
- HuggingFace简介

深度学习框架

深度学习模型在很多任务上十分有效。

从头开始实现一个深度学习模型？

- 需要几天（甚至几周）才能得到一个有效的模型，时间成本太高！

AlexNet是首个在大规模图像识别问题取得突破性进展的深度神经网络 （2012年）

深度学习框架

- 什么是深度学习框架？
  - 深度学习框架是一种**接口**、**库**或**工具**，利用**预先构建和优化好的组件**集合定义模型。

- 特性
  - 性能优化
  - 易于理解和编码
  - 良好的社区支持
  - 并行处理
  - 自动计算梯度

核心组件

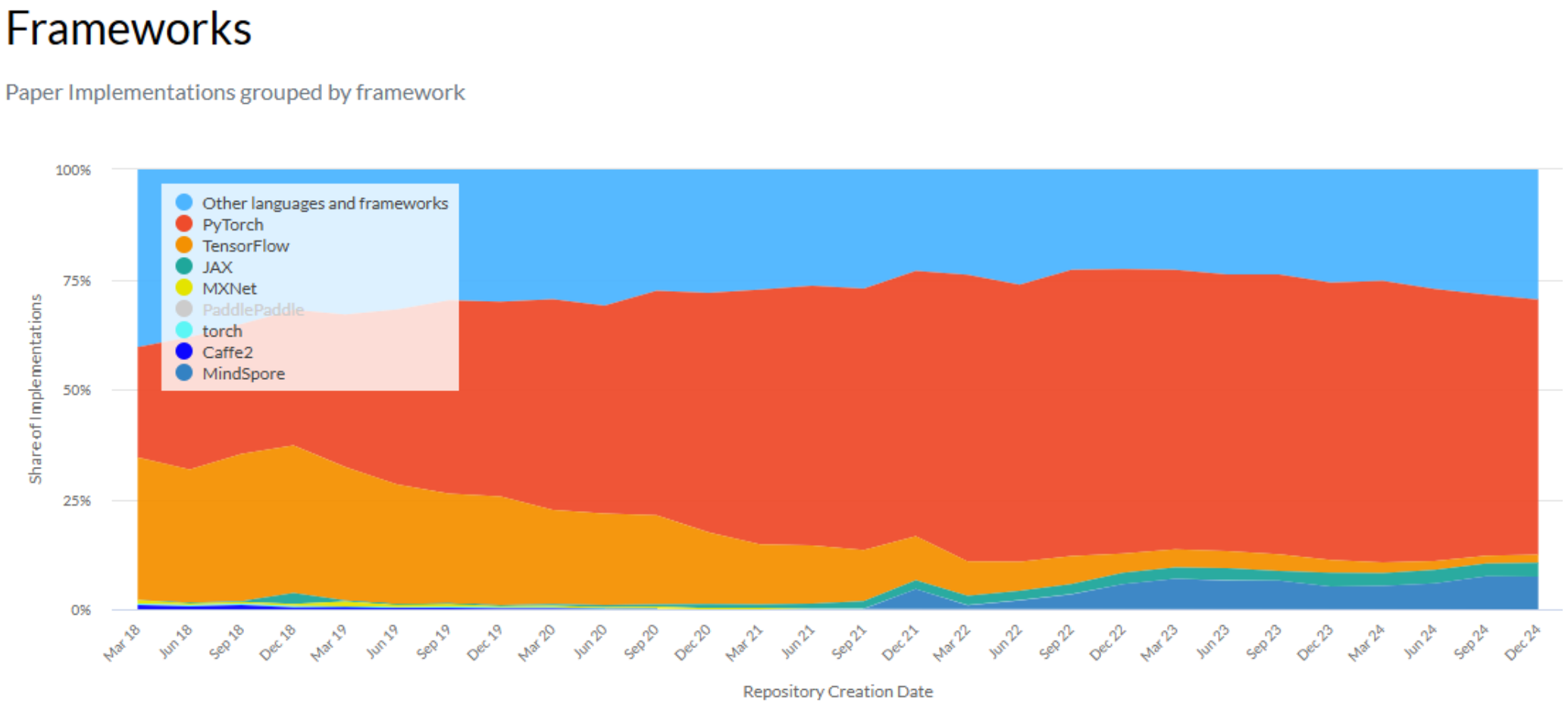
- 张量
- 基于张量的操作
- 计算图
- 自动微分工具
- BLAS、 cuBLAS、 cuDNN等拓展包

Tensorflow与PyTorch比较分析

- 目前来看TensorFlow和PyTorch框架是业界使用最为广泛的两个深度学习框架
  - TensorFlow在工业界拥有完备的解决方案和用户基础
  - PyTorch 得益于其精简灵活的接口设计，可以快速设计和调试网络模型，在学术界广受好评。

Tensorflow与PyTorch比较分析

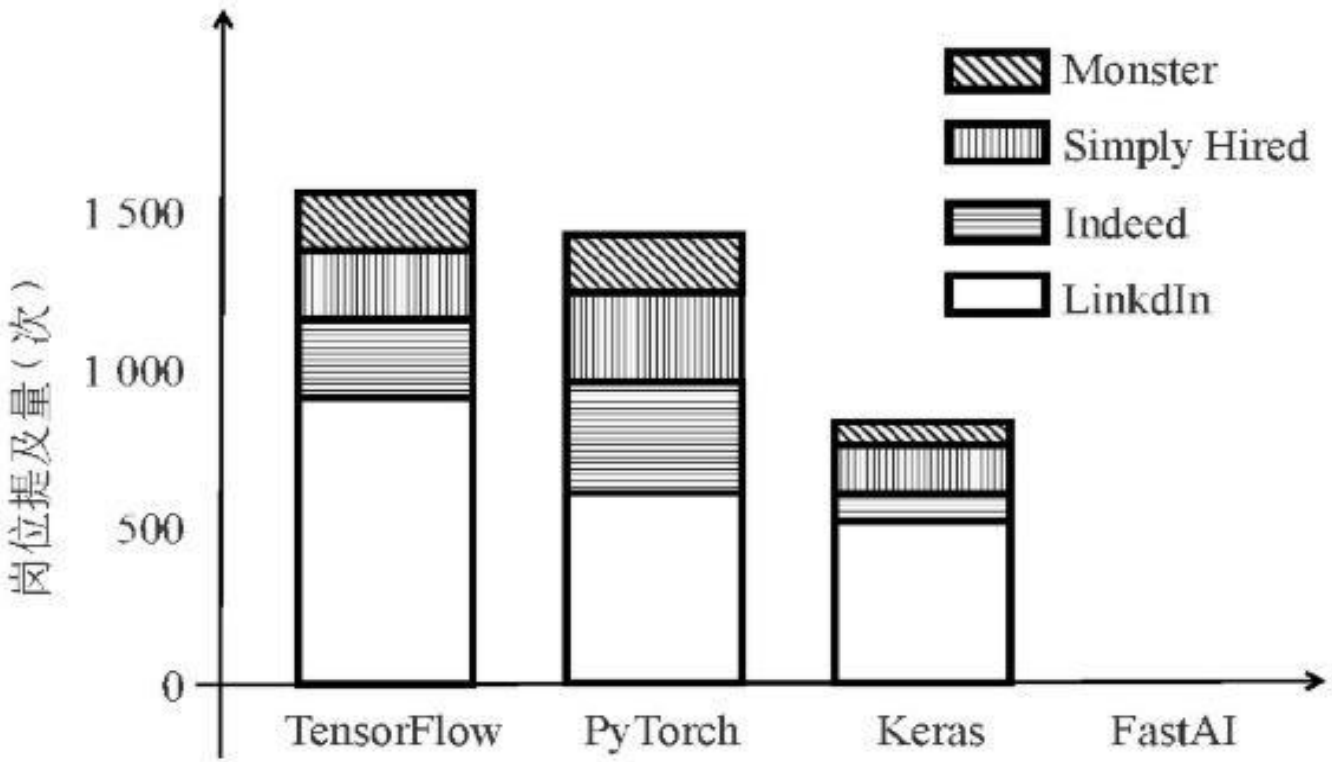
- 学术界：Github的研究项目上使用Tensorflow, PyTorch和其他框架比例



Source: https://paperswithcode.com/trends

Tensorflow与PyTorch比较分析

- **工业界：**线上招聘启事中的提及率



Source: HE H. The State of Machine Learning Frameworks in 2019.(2019-10-10).https://thegradient.pub/state-of-ml-frameworks-2019-pytorch- dominates-research-tensorflow-dominates-industry/.

PyTorch入门

- PyTorch是一个基于Python的库，用来提供一个具有灵活性的深度学习开发平台。PyTorch的工作流程非常接近Python的科学计算库——numpy

- PyTorch的其他一些优点还包括：多gpu支持， 自定义数据加载器和简化的预处理器。

PyTorch入门-Tensor和autograd

- Tensor
  - Tensor， 又名张量。 可简单地认为它是一个数组， 且支持高效的科学计算。 它可以是一个数（标量） 、 一维数组（ 向量） 、 二维数组（ 矩阵） 或更高维的数组（ 高阶数据） 。 Tensor和numpy的ndarrays类似， 但PyTorch的tensor支持GPU加速。 如下定义一个简单的一维矩阵：

```
import torch
torch.FloatTensor([2])
```

  - 与numpy一样，科学计算库非常重要的一点是能够实现高效的数学功能。如下实现一个简单的加法操作：

```
a = torch.FloatTensor([2])
b = torch.FloatTensor([3])
a + b
```

PyTorch入门-Tensor和autograd

- autograd
  - torch.autograd是为了方便用户使用， 专门开发的一套自动求导引擎， 它能够根据输入和前向传播过程自动构建计算图， 并执行反向传播。

```python
import torch


x = torch.tensor([2.0], requires_grad=True)


def f(x):
    return 5 * x ** 2 + 2 * x - 1


# 计算函数的输出
y = f(x)

# 使用backward()方法计算梯度
y.backward()

# 获取梯度值
gradient = x.grad
print(gradient)
```

$$y' = 10x + 2$$
$$22 = 10 * 2 + 2$$

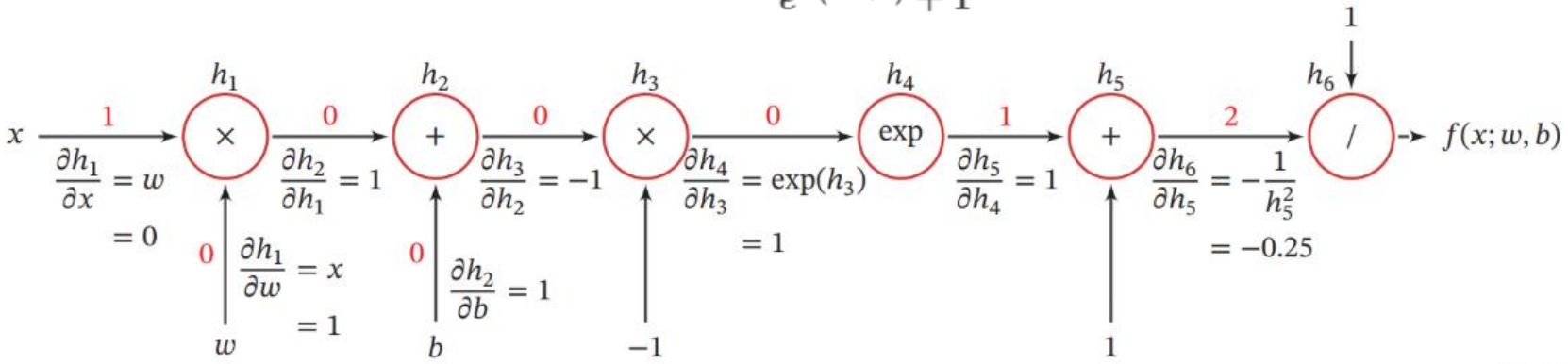
PyTorch入门-Tensor和autograd

- 计算图：计算图是一种特殊的有向无环图， 用于记录算子与变量之间的关系。
- 在深度学习中，计算图帮助我们理解模型的训练过程，直观地把握损失函数对模型参数的影响，同时为反向传播算法提供了理论基础。

PyTorch入门-Tensor和autograd

- 计算图：计算图是一种特殊的有向无环图， 用于记录算子与变量之间的关系。
- 一个完整的计算图可以分为正向传播和反向传播两个阶段：
  - 正向传播（Forward Propagation）：输入数据经过计算节点逐层传播，最终得到输出结果。
  - 反向传播（Backward Propagation）：首先根据损失函数计算输出结果与真实标签之间的误差，然后利用链式法则，逐个计算每个计算节点对应的输入的梯度，最终得到参数的梯度信息。

$$f(x;w,b)=\frac{1}{e^{-(wx+b)}+1}$$



PyTorch入门-Tensor和autograd

- 计算图：计算图是一种特殊的有向无环图， 用于记录算子与变量之间的关系。

函数	导数	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

PyTorch入门-nn

- nn.Module
  - torch.nn的核心数据结构是Module， 它是一个抽象的概念， 既可以表示神经网络中的某个层， 也可以表示一个包含很多层的神经网络。 最常见的做法就是继承nn.Module， 编写自己的网络。能够自动检测到自己的parameter， 并将其作为学习参数。
  - 主Module能够递归查找子Module中的parameter。
  - 为了方便用户使用， PyTorch实现了神经网络中绝大多数的layer， 这些layer都继承于nn.Module， 封装了可学习参数parameter， 并实现了forward函数， 且专门针对GPU运算进行了CuDNN优化， 其速度和性能都十分优异。
    - 可学习的参数：卷积层和全连接层的权重、 bias、 BatchNorm的γ、 β等。
    - 不可学习的参数(超参数)：学习率、 batch size、 weight decay、模型的深度宽度等。

PyTorch入门-nn

- nn.Module
  - 具体层可参照官方文档（http://pytorch.org/docs/nn.html）
  - 阅读文档时应主要关注以下几点：
    - 构造函数的参数，如nn.Linear(in_features, out_features, bias)，需关注这三个参数的作用。
    - 属性、可学习参数和子module。如nn.Linear中有weight和bias两个可学习参数，不包含子module。
    - 输入输出的形状，如nn.linear的输入形状是(N, input_features)，输出为(N, output_features)， N是batch_size。

PyTorch入门-nn

- 要实现一个自定义层大致分以下几个主要的步骤：
  - 自定义一个类，**继承自Module类**，并且一定要实现两个基本的函数
    - 构造函数__init__
    - 前向计算函数forward函数
  - 在构造函数__init__中实现层的参数定义。
    - 例如：Linear层的权重和偏置
  - 在前向传播forward函数里面实现前向运算。
    - 通过torch.nn.functional函数来实现
    - 自定义自己的运算方式。如果该层含有权重，那么权重必须是nn.Parameter类型

PyTorch入门-nn

- 自定义的简单例子：y=**w***sqrt(X²+**bias**)

```
import torch

class MyLayer(torch.nn.Module):
    def __init__(self, in_features, out_features):
        super(MyLayer, self).__init__() # 和自定义模型一样，第一句话就是调用父类的构造函数
        self.in_features = in_features
        self.out_features = out_features
        self.weight = torch.nn.Parameter(torch.Tensor(in_features, out_features)) # 由于weights是可以训练的，所以使用Parameter来定义。
        self.bias = torch.nn.Parameter(torch.Tensor(in_features)) # 由于bias是可以训练的，所以使用Parameter来定义
        # 初始化参数数值
        torch.nn.init.uniform_(self.weight)
        torch.nn.init.uniform_(self.bias)
    def forward(self, x):
        input_=torch.pow(x,2)+self.bias
        input_=torch.sqrt(input_)
        y=torch.matmul(input_,self.weight)
        return y

class MyNet(torch.nn.Module):
    def __init__(self, in_features, out_features):
        super(MyNet, self).__init__()  # 第一句话，调用父类的构造函数
        self.mylayer1 = MyLayer(in_features, out_features)
    def forward(self, x):
        x = self.mylayer1(x)
        return x

N, D_in, D_out = 10, 10, 1   # 一共10组样本，输入特征为10，输出特征为1
model = MyNet(D_in, D_out)  # 先定义一个模型
print(model)
```

```
MyNet(
  (mylayer1): MyLayer()
)
```

PyTorch入门-nn

- 构建模型需要注意：
  - 如果模型有可学习的参数时，最好使用nn.Module
  - 卷积、全连接等有可学习参数的网络建议使用nn.Module
  - 激活函数（ReLu、sigmoid、Tanh）、池化（MaxPool）等层没有可学习的参数，可以使用对应的functional函数

PyTorch入门-nn 损失函数（Loss Function）

- 损失函数（loss function）是用来估量模型的预测值$f(x)$与真实值Y的不一致程度，它是一个非负实值函数,通常使用$L(Y, f(x))$来表示，损失函数越小，模型的表现就越好。

- 常见的损失函数：
  - 分类任务：cross entropy loss; the Binary Cross Entropy;
  - 回归任务：mean absolute error; mean squared error;

https://pytorch.org/docs/stable/nn.html#loss-functions

PyTorch入门-nn 损失函数（Loss Function）

Loss Functions

`nn.L1Loss`	Creates a criterion that measures the mean absolute error (MAE) between each element in the input x and target y.
`nn.MSELoss`	Creates a criterion that measures the mean squared error (squared L2 norm) between each element in the input x and target y.
`nn.CrossEntropyLoss`	This criterion computes the cross entropy loss between input logits and target.
`nn.CTCLoss`	The Connectionist Temporal Classification loss.
`nn.NLLLoss`	The negative log likelihood loss.
`nn.PoissonNLLLoss`	Negative log likelihood loss with Poisson distribution of target.
`nn.GaussianNLLLoss`	Gaussian negative log likelihood loss.
`nn.KLDivLoss`	The Kullback-Leibler divergence loss.
`nn.BCELoss`	Creates a criterion that measures the Binary Cross Entropy between the target and the input probabilities:
`nn.BCEWithLogitsLoss`	This loss combines a *Sigmoid* layer and the *BCELoss* in one single class.

L1Loss

CLASS torch.nn.L1Loss(*size_average=None, reduce=None, reduction='mean'*) [SOURCE]

Creates a criterion that measures the mean absolute error (MAE) between each element in the input x and target y.

The unreduced (i.e. with `reduction` set to `'none'`) loss can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = |x_n - y_n|,$$

where N is the batch size. If `reduction` is not `'none'` (default `'mean'`), then:

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction} = \text{'mean'}; \\ \text{sum}(L), & \text{if reduction} = \text{'sum'}. \end{cases}$$

x and y are tensors of arbitrary shapes with a total of N elements each.

The sum operation still operates over all the elements, and divides by N.

The division by N can be avoided if one sets `reduction = 'sum'`.

Supports real-valued and complex-valued inputs.

https://pytorch.org/docs/stable/nn.html#loss-functions

PyTorch入门-nn 损失函数（Loss Function）

```
loss_function = torch.nn.L1Loss()
predicted_value = torch.randn(3, 5, requires_grad=True) # 创建一个结构为（3, 5）的随机数值Tensor
target_value = torch.randn(3, 5) # 创建一个结构为（3, 5）的随机数值Tensor
loss = loss_function(predicted_value, target_value)
loss.backward()
print(predicted_value)
print('\n')
print(target_value)
print('\n')
print(loss)
```

```
tensor([[ 1.0977, -0.9135, -0.3732, -1.2982,  0.4719],
        [-1.6926,  0.0513, -1.5262, -0.7715,  0.3461],
        [ 0.5225, -0.7372,  0.6265, -0.8965,  0.3871]], requires_grad=True)


tensor([[ 2.9978, -1.5556, -0.1066, -1.5669, -0.7586],
        [-0.1973,  0.0639,  0.6549, -0.1207,  0.2146],
        [-0.4125,  0.5580,  0.1840, -0.4663, -0.1532]])


tensor(0.8282, grad_fn=<MeanBackward0>)
```


https://pytorch.org/docs/stable/nn.html#loss-functions

PyTorch入门-优化器和学习率

- 优化器
  - PyTorch将深度学习中常用的优化方法全部封装在torch.optim中，其设计十分灵活，能够很方便地扩展成自定义的优化方法。
  - 所有的优化方法都是继承基类optim.Optimizer，并实现了自己的优化步骤。最基本的优化方法是随机梯度下降法（SGD）。
  - 需要重点关注的是，优化时如何调整学习率。
- 调整学习率
  - 修改optimizer.param_groups中对应的学习率。（一般由特定算法调整）
  - 新建优化器，由于optimizer十分轻量级，构建开销很小，故可以构建新的optimizer。

PyTorch入门-优化器和学习率

- 优化器样例

torch.optim

`torch.optim` is a package implementing various optimization algorithms.

Most commonly used methods are already supported, and the interface is general enough, so that more sophisticated ones can also be easily integrated in the future.

How to use an optimizer

To use `torch.optim` you have to construct an optimizer object that will hold the current state and will update the parameters based on the computed gradients.

Constructing it

To construct an `Optimizer` you have to give it an iterable containing the parameters (all should be `Parameter` s) or named parameters (tuples of (str, `Parameter`)) to optimize. Then, you can specify optimizer-specific options such as the learning rate, weight decay, etc.

Example:

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
optimizer = optim.Adam([var1, var2], lr=0.0001)
```

Named parameters example:

```
optimizer = optim.SGD(model.named_parameters(), lr=0.01, momentum=0.9)
optimizer = optim.Adam([('layer0', var1), ('layer1', var2)], lr=0.0001)
```


Source: https://pytorch.org/docs/stable/optim.html

PyTorch入门-优化器和学习率

优化器样例：SGD

SGD

CLASS torch.optim.SGD(*params*, *lr=0.001*, *momentum=0*, *dampening=0*, *weight_decay=0*, *nesterov=False*, **, maximize=False*, *foreach=None*, *differentiable=False*, *fused=None*) [SOURCE]

Implements stochastic gradient descent (optionally with momentum).

$$
\begin{aligned}
&\textbf{input}: \gamma \text{ (lr)}, \ \theta_0 \text{ (params)}, \ f(\theta) \text{ (objective)}, \ \lambda \text{ (weight decay)}, \\
&\qquad \mu \text{ (momentum)}, \ \tau \text{ (dampening)}, \ \textit{nesterov}, \ \textit{maximize} \\
&\textbf{for } t = 1 \textbf{ to } \ldots \textbf{ do} \\
&\quad g_t \leftarrow \nabla_\theta f_t(\theta_{t-1}) \\
&\quad \textbf{if } \lambda \neq 0 \\
&\quad\quad g_t \leftarrow g_t + \lambda \theta_{t-1} \\
&\quad \textbf{if } \mu \neq 0 \\
&\quad\quad \textbf{if } t > 1 \\
&\quad\quad\quad \mathbf{b}_t \leftarrow \mu \mathbf{b}_{t-1} + (1-\tau) g_t \\
&\quad\quad \textbf{else} \\
&\quad\quad\quad \mathbf{b}_t \leftarrow g_t \\
&\quad\quad \textbf{if } \textit{nesterov} \\
&\quad\quad\quad g_t \leftarrow g_t + \mu \mathbf{b}_t \\
&\quad\quad \textbf{else} \\
&\quad\quad\quad g_t \leftarrow \mathbf{b}_t \\
&\quad \textbf{if } \textit{maximize} \\
&\quad\quad \theta_t \leftarrow \theta_{t-1} + \gamma g_t \\
&\quad \textbf{else} \\
&\quad\quad \theta_t \leftarrow \theta_{t-1} - \gamma g_t \\
&\textbf{return } \theta_t
\end{aligned}
$$

Nesterov momentum is based on the formula from On the importance of initialization and momentum in deep learning.

Parameters

- **params** (*iterable*) – iterable of parameters or named_parameters to optimize or iterable of dicts defining parameter groups. When using named_parameters, all parameters in all groups should be named
- **lr** (*float, Tensor, optional*) – learning rate (default: 1e-3)
- **momentum** (*float, optional*) – momentum factor (default: 0)
- **dampening** (*float, optional*) – dampening for momentum (default: 0)
- **weight_decay** (*float, optional*) – weight decay (L2 penalty) (default: 0)
- **nesterov** (*bool, optional*) – enables Nesterov momentum. Only applicable when momentum is non-zero. (default: False)
- **maximize** (*bool, optional*) – maximize the objective with respect to the params, instead of minimizing (default: False)
- **foreach** (*bool, optional*) – whether foreach implementation of optimizer is used. If unspecified by the user (so foreach is None), we will try to use foreach over the for-loop implementation on CUDA, since it is usually significantly more performant. Note that the foreach implementation uses ~ sizeof(params) more peak memory than the for-loop version due to the intermediates being a tensorlist vs just one tensor. If memory is prohibitive, batch fewer parameters through the optimizer at a time or switch this flag to False (default: None)
- **differentiable** (*bool, optional*) – whether autograd should occur through the optimizer step in training. Otherwise, the step() function runs in a torch.no_grad() context. Setting to True can impair performance, so leave it False if you don't intend to run autograd through this instance (default: False)
- **fused** (*bool, optional*) – whether the fused implementation is used. Currently, *torch.float64*, *torch.float32*, *torch.float16*, and *torch.bfloat16* are supported. (default: None)

https://pytorch.org/docs/stable/generated/torch.optim.SGD.html

PyTorch入门-优化器和学习率

How to adjust learning rate

`torch.optim.lr_scheduler.LRScheduler` provides several methods to adjust the learning rate based on the number of epochs. `torch.optim.lr_scheduler.ReduceLROnPlateau` allows dynamic learning rate reducing based on some validation measurements.

Learning rate scheduling should be applied after optimizer's update; e.g., you should write your code this way:

Example:

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
scheduler = ExponentialLR(optimizer, gamma=0.9)

for epoch in range(20):
    for input, target in dataset:
        optimizer.zero_grad()
        output = model(input)
        loss = loss_fn(output, target)
        loss.backward()
        optimizer.step()
    scheduler.step()
```

Most learning rate schedulers can be called back-to-back (also referred to as chaining schedulers). The result is that each scheduler is applied one after the other on the learning rate obtained by the one preceding it.

Example:

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
scheduler1 = ExponentialLR(optimizer, gamma=0.9)
scheduler2 = MultiStepLR(optimizer, milestones=[30,80], gamma=0.1)

for epoch in range(20):
    for input, target in dataset:
        optimizer.zero_grad()
        output = model(input)
        loss = loss_fn(output, target)
        loss.backward()
        optimizer.step()
    scheduler1.step()
    scheduler2.step()
```

Some schedulers

`lr_scheduler.LRScheduler`	Adjusts the learning rate during optimization.
`lr_scheduler.LambdaLR`	Sets the initial learning rate.
`lr_scheduler.MultiplicativeLR`	Multiply the learning rate of each parameter group by the factor given in the specified function.
`lr_scheduler.StepLR`	Decays the learning rate of each parameter group by gamma every step_size epochs.
`lr_scheduler.MultiStepLR`	Decays the learning rate of each parameter group by gamma once the number of epoch reaches one of the milestones.
`lr_scheduler.ConstantLR`	Multiply the learning rate of each parameter group by a small constant factor.
`lr_scheduler.LinearLR`	Decays the learning rate of each parameter group by linearly changing small multiplicative factor.
`lr_scheduler.ExponentialLR`	Decays the learning rate of each parameter group by gamma every epoch.
`lr_scheduler.PolynomialLR`	Decays the learning rate of each parameter group using a polynomial function in the given total_iters.
`lr_scheduler.CosineAnnealingLR`	Set the learning rate of each parameter group using a cosine annealing schedule.

https://pytorch.org/docs/stable/optim.html

PyTorch入门-优化器和学习率

```
# 创建输入、输出数据
x = torch.randn(N, D_in)  #（10, 10）
y = torch.randn(N, D_out) #（10, 1）
loss_fn = torch.nn.MSELoss(reduction='mean')
learning_rate = 1e-3  #定义学习率
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)  #构造一个optimizer对象

for t in range(10):
    # 第一步：数据的前向传播，计算预测值p_pred
    y_pred = model(x)
    # 第二步：计算计算预测值p_pred与真实值的误差
    loss = loss_fn(y_pred, y)
    print(f"第 {t} 个Epoch, 损失是 {loss.item()}")
    # 在反向传播之前，将模型的梯度归零
    optimizer.zero_grad()
    # 第三步：反向传播误差
    loss.backward()
    # 直接通过梯度一步到位，更新完整个网络的训练参数
    optimizer.step()
```

```
第 0 个Epoch, 损失是 21.833877563476562
第 1 个Epoch, 损失是 21.716915130615234
第 2 个Epoch, 损失是 21.600379943847656
第 3 个Epoch, 损失是 21.484264373779297
第 4 个Epoch, 损失是 21.368587493896484
第 5 个Epoch, 损失是 21.253345489501953
第 6 个Epoch, 损失是 21.1385498046875
第 7 个Epoch, 损失是 21.02420425415039
第 8 个Epoch, 损失是 20.91031265258789
第 9 个Epoch, 损失是 20.796884536743164
```

PyTorch入门-常用工具

- 数据处理：
  - 在PyTorch中，数据加载可通过自定义的数据集对象实现。数据集对象被抽象为Dataset类，实现自定义的数据集需要继承Dataset，并实现两个Python方法。
  - __getitem__：返回一条数据或一个样本。
  - __ len__：返回样本的数量
  - Dataset只负责数据的抽象，**一次调用__getitem__**只返回一个样本。在训练神经网络时，是对一个batch的数据进行操作，同时还需要对数据进行并行加速等。因此，PyTorch提供了DataLoader帮助实现这些功能。

PyTorch入门-常用工具

- 数据处理 Dataset：

Creating a Custom Dataset for your files

A custom Dataset class must implement three functions: *__init__*, *__len__*, and *__getitem__*. Take a look at this implementation; the FashionMNIST images are stored in a directory `img_dir`, and their labels are stored separately in a CSV file `annotations_file`.

In the next sections, we'll break down what's happening in each of these functions.

```
import os
import pandas as pd
from torchvision.io import read_image

class CustomImageDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None, target_transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = target_transform

    def __len__(self):
        return len(self.img_labels)

    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 0])
        image = read_image(img_path)
        label = self.img_labels.iloc[idx, 1]
        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            label = self.target_transform(label)
        return image, label
```

Source: https://pytorch.org/tutorials/beginner/basics/data_tutorial.html

PyTorch入门-常用工具

- DataLoader函数定义如下：

DataLoader(dataset, batch_size=1, shuffle=False, sampler=None, num_workers=0, collate_fn=default_collatem, pin_memory=False, drop_last=False)

- 参数解释如下：
  - dataset：加载的数据集（Dataset对象）
  - batch_size：batch size（批大小）
  - shuffle：是否将数据打乱
  - sampler：样本抽样
  - num_workers：使用多进程加载的进程数，0代表不使用多进程
  - collate_fn：如何将多个样本数据拼接成一个batch，一般使用默认的拼接方式即可
  - pin_memory：是否将数据保存在pin memory区，pin memory中的数据转到GPU会快一些
  - drop_last：dataset中的数据个数可能不是batch_size的整数倍，drop_last为True会将多出来不足一个batch的数据丢弃

Source: https://pytorch.org/tutorials/beginner/basics/data_tutorial.html

PyTorch入门-常用工具

- 在PyTorch中以下数据结构分为CPU和GPU两个版本。
  - Tensor
  - Variable（包括Parameter）
  - nn.Module（包括常用的layer、loss function，以及容器sequential等）

- 使用GPU加速

它们都带有一个.cuda方法，调用此方法即可将其转化为对应的GPU对象。如果服务器具有 多个GPU，tensor.cuda()方法会将tensor保存到第一块GPU上，等价于tensor.cuda(0)。此时如果想使用第二块GPU，需要手动指定tensor.cuda(1)

PyTorch入门-常用工具

- 使用GPU加速
  - 调用torch.cuda.set_device(1)指定使用第二块GPU，后续的.cuda()都无需更改，切换GPU只需修改这一行代码。
  - 设置环境变量CUDA_VISIBLE_DEVICES
    - export CUDA_VISIBLE_DEVICE=1时（**下标是从0开始，1代表第二块GPU**）
    - CUDA_VISIBLE_DEVICES还可以指定多个GPU，如：export CUDA_VISIBLE_DEVICES = 0,2,3
  - 在命令行运行代码时，指定使用的GPU
    - CUDA_VISIBLE_DEVICE=0 python train_model.py

PyTorch入门-常用工具

- 使用GPU加速，常见问题
  - 数据、模型不在同一个计算设备上
    - 比如，模型加载在GPU，数据加载在CPU
    - 比如，模型加载在第一个GPU，数据加载在第二个GPU
  - 显存不足：RuntimeError: CUDA error: out of memory
    - 模型、数据太大，显卡显存不足以支持
    - 解决方案：1. 升级设备； 2. 优化代码，及时释放无用的模型、数据等等
  - 分布式训练
    - 分布式训练需要对代码进行调整
    - 解决方案：参考 torch.nn.parallel.DistributedDataParallel 接口； 使用已经封装好的库，如huggingface

PyTorch入门-常用工具

- 保存和加载
  - 在PyTorch中，需要将Tensor、Variable等对象保存到硬盘，以便后续能通过相应的方法加载到内存中。
    - 这些信息最终都是保存成Tensor。
    - 使用torch.save和torch.load即可完成Tensor保存和加载的功能。

  - 使用方式：
    - 保存模型的参数：torch.save(model.state_dict(), PATH)
    - 读取保存的模型参数：model.load_state_dict(torch.load(PATH, weights_only=True))

PyTorch入门—样例

- 使用PyTorch构建一个分类模型
- 常见的流程：
  - 数据读取、处理
  - 模型初始化
  - 训练参数、配置
  - 训练模型
  - 验证模型
  - 保存模型
  - 应用模型
- 样例代码见”week1.ipynb”
- 请尝试在你的电脑配置环境，并且运行样例代码

Hugging Face

- Hugging Face是一家美国公司，专门开发用于构建机器学习应用的工具。 该公司的代表产品是其为自然语言处理应用构建的**transformers库**，以及允许用户共享机器学习模型和数据集的**平台**。
- 一个非常强大的平台！本门课程后续的复杂模型会基于transformers库和Hugging Face提供的平台。

🤗 Transformers简介

为 PyTorch、TensorFlow 和 JAX 打造的先进的机器学习工具.

🤗 Transformers 提供了可以轻松地下载并且训练先进的预训练模型的 API 和工具。使用预训练模型可以减少计算消耗和碳排放，并且节省从头训练所需要的时间和资源。这些模型支持不同模态中的常见任务，比如：

📝 **自然语言处理**：文本分类、命名实体识别、问答、语言建模、摘要、翻译、多项选择和文本生成。
🖼️ **机器视觉**：图像分类、目标检测和语义分割。
🗣️ **音频**：自动语音识别和音频分类。
🐙 **多模态**：表格问答、光学字符识别、从扫描文档提取信息、视频分类和视觉问答。

🤗 Transformers 支持在 PyTorch、TensorFlow 和 JAX 上的互操作性. 这给在模型的每个阶段使用不同的框架带来了灵活性；在一个框架中使用几行代码训练一个模型，然后在另一个框架中加载它并进行推理。模型也可以被导出为 ONNX 和 TorchScript 格式，用于在生产环境中部署。

https://huggingface.co/docs/transformers/zh/index

Hugging Face

创建一个 pipeline() 实例并且指定你想要将它用于的任务，就可以开始了。你可以将 pipeline() 用于任何一个上面提到的任务，如果想知道支持的任务的完整列表，可以查阅 pipeline API 参考。不过，在这篇教程中，你将把 pipeline() 用在一个情感分析示例上：

```
>>> from transformers import pipeline

>>> classifier = pipeline("sentiment-analysis")
```

pipeline() 会下载并缓存一个用于情感分析的默认的预训练模型和分词器。现在你可以在目标文本上使用 `classifier` 了：

```
>>> classifier("We are very happy to show you the 🤗 Transformers library.")
[{'label': 'POSITIVE', 'score': 0.9998}]
```

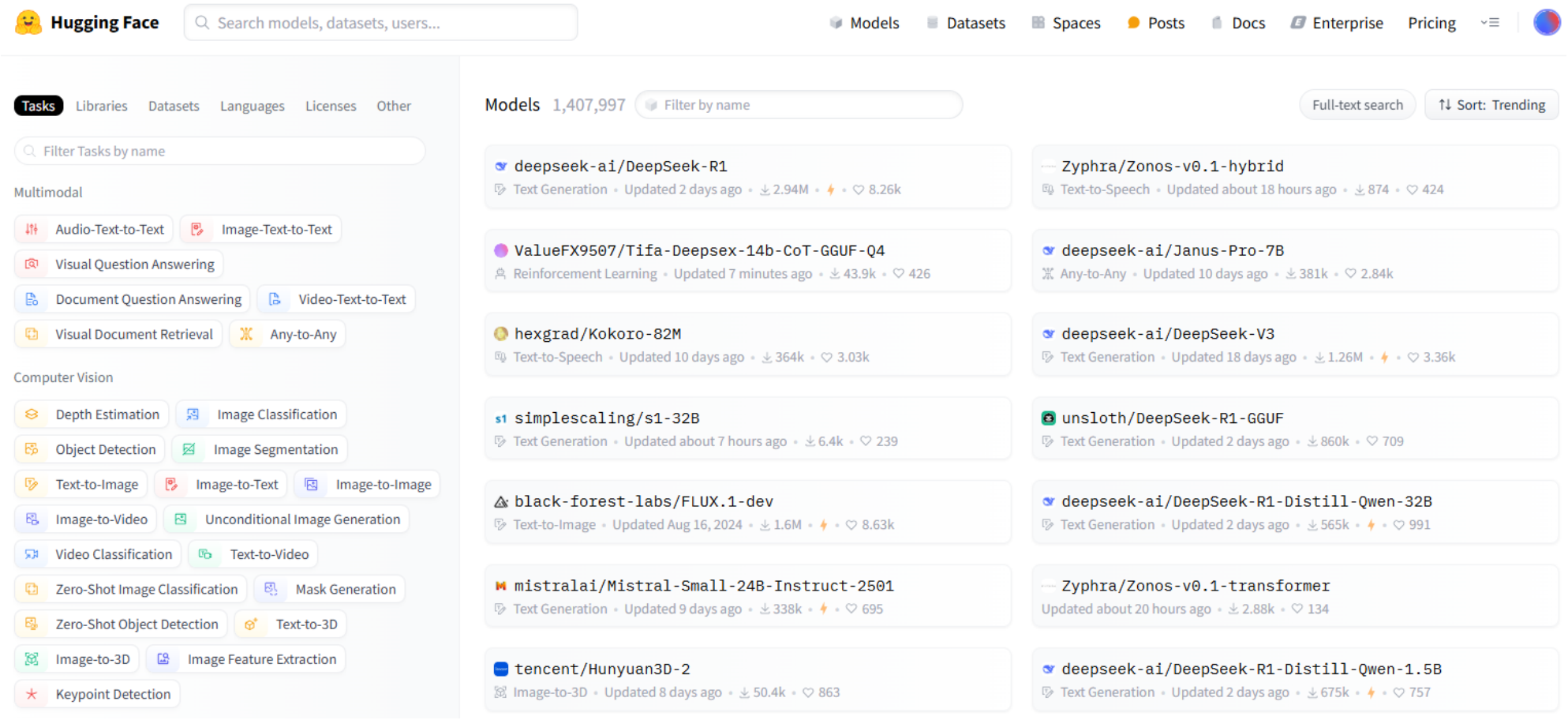
如果你有不止一个输入，可以把所有输入放入一个列表然后传给pipeline()，它将会返回一个字典列表：

```
>>> results = classifier(["We are very happy to show you the 🤗 Transformers library.", "We hope you don
>>> for result in results:
...     print(f"label: {result['label']}, with score: {round(result['score'], 4)}")
label: POSITIVE, with score: 0.9998
label: NEGATIVE, with score: 0.5309
```


https://huggingface.co/docs/transformers/zh/index

Hugging Face

- 可以通过Hugging Face获取其他用户分享的模型或数据集



https://huggingface.co/models

课后准备

- 在自己电脑配置anaconda, Python环境
- 根据自身条件，至少安装、熟悉一个编程工具
  - 国内可以使用Trae
  - 条件允许的话，可使用cursor

作业1

- 作业1：注册deepseek API; 使用Coding Agent和deepseek API完成基础的文本分析任务
- 第三周上课前提交